CUDA

December 17, 2025

Ramasamy Kandasamy

Function Qualifiers

`__global__`	Kernel func. Returns void.
`__device__`	Function callable only from device.
`__host__`	Function callable from host (default).
`__host__ __device__`	Compiled for both host and device.

Kernel Launch:

```
kernel<<<gridDim, blockDim, sharedMem, stream>>>(args);
```

`gridDim`	Grid size (int or dim3).
`blockDim`	Block size (int or dim3).
`sharedMem`	Dynamic shared memory bytes (optional).
`stream`	CUDA stream (optional).

`__device__` To declare global variable in the device.

Built-in Variables

`gridDim`	Dimensions of the grid (`dim3`).
`gridDim.x/y/z`	Number of blocks in each dimension.
`blockDim`	Dimensions of each block (`dim3`).
`blockDim.x/y/z`	Number of threads in each dimension.
`blockIdx`	Block index within the grid (`uint3`).
`blockIdx.x/y/z`	Current block's index in each dimension.
`threadIdx`	Thread index within the block (`uint3`).
`threadIdx.x/y/z`	Current thread's index in each dimension.
`warpSize`	Threads per warp (constant = 32).

Global Thread ID:

```
int tid = blockIdx.x * blockDim.x + threadIdx.x; // 1D
int col = blockIdx.x * blockDim.x + threadIdx.x; // 2D
int row = blockIdx.y * blockDim.y + threadIdx.y;
```

Memory Management

`cudaMalloc(&ptr, size)`	Allocate device memory. NOTE: ptr to ptr.
`cudaFree(ptr)`	Free device memory.
`cudaHostAlloc(&ptr, size, flags)`	Pinned host memory (faster transfers)
`cudaFreeHost(ptr)`	Free pinned host memory
`cudaMallocManaged(&ptr, size)`	Unified memory (host & device accessible)

`cudaMemcpy(dst, src, size, kind)`

`dst`	Pointers.
`src`	
kind	`cudaMemcpyHostToDevice`.
	`cudaMemcpyDeviceToHost`.
	`cudaMemcpyDeviceToDevice`.
`size`	Eg: `n * sizeof(int)`.

`cudaMemcpyAsync(..., stream)`	Async copy. Requires pinned memory

Memory Types

Type	Scope	Description
Registers	Thread	Fastest; automatic allocation.
Local	Thread	Spills from registers; stored in global memory.
Shared	Block	Fast; `__shared__` qualifier.
Global	Grid	Slowest; `cudaMalloc` or `__device__`.
Constant	Grid	Read-only; cached; `__constant__` (64KB max).
Texture	Grid	Read-only; cached; optimized for 2D spatial locality.

Synchronization

Synchronization Primitives

Function	Description
`__syncthreads()`	Block-level barrier; all threads in block must reach.
`__syncwarp(mask)`	Warp-level sync; mask specifies participating threads.
`cudaDeviceSynchronize()`	Host waits for all device work to complete.
`cudaStreamSynchronize(s)`	Host waits for all work in stream `s`.
`cudaEventSynchronize(e)`	Host waits for event `e` to complete.

Memory Fences

`__threadfence_block()`	Ensures writes visible to block threads.
`__threadfence()`	Ensures writes visible to all device threads.
`__threadfence_system()`	Ensures writes visible to host and device.

Atomic Operations

Function	Operation
`atomicAdd(addr, val)`	`*addr += val`; returns old value.
`atomicSub(addr, val)`	`*addr -= val`; returns old value.
`atomicExch(addr, val)`	`*addr = val`; returns old value.
`atomicMin(addr, val)`	`*addr = min(*addr, val)`.
`atomicMax(addr, val)`	`*addr = max(*addr, val)`.
`atomicInc(addr, val)`	`*addr = (*addr >= val) ? 0 : *addr+1`.
`atomicDec(addr, val)`	`*addr = (*addr==0 || *addr>val) ? val : *addr-1`.
`atomicCAS(addr, cmp, val)`	If `*addr == cmp`, set `*addr = val`; returns old.

Bitwise Atomics

`atomicAnd(addr, val)`	`*addr &= val`.
`atomicOr(addr, val)`	`*addr |= val`.
`atomicXor(addr, val)`	`*addr ^= val`.

Streams

`cudaStream_t`	Stream handle type.
`cudaStreamCreate(&s)`	Create a stream.
`cudaStreamDestroy(s)`	Destroy a stream.

`cudaStreamCreateWithFlags(&s, flags)`

`cudaStreamDefault`	Default stream behavior.
`cudaStreamNonBlocking`	No implicit sync with default stream.

Kernel launch on stream:

```
kernel<<<grid, block, sharedMem, stream>>>(args);
```

`cudaStreamQuery(s)`	Returns `cudaSuccess` if stream idle.
`cudaStreamSynchronize(s)`	Host blocks until stream completes.
`cudaStreamWaitEvent(s, e, 0)`	Stream waits for event `e`.

Events

`cudaEvent_t`	Event handle type.
`cudaEventCreate(&e)`	Create an event.
`cudaEventDestroy(e)`	Destroy an event.
`cudaEventRecord(e, s)`	Record event in stream `s`.
`cudaEventSynchronize(e)`	Host blocks until event completes.
`cudaEventQuery(e)`	Returns `cudaSuccess` if event complete.

Timing with events:

```
cudaEventRecord(start, stream);
kernel<<<...>>>();
cudaEventRecord(stop, stream);
cudaEventSynchronize(stop);
float ms;
cudaEventElapsedTime(&ms, start, stop);
```

Stream Priorities

`cudaDeviceGetStreamPriorityRange(&lo, &hi)`	Get priority range.
`cudaStreamCreateWithPriority(&s, flags, pri)`	Create with priority.

- Lower number = higher priority.
- Work from higher-priority streams may preempt lower.

Support

Error Handling

`cudaError_t`	Error type returned by CUDA calls.
`cudaSuccess`	No error (value 0).
`cudaGetLastError()`	Returns and resets last error.
`cudaPeekAtLastError()`	Returns last error without reset.
`cudaGetErrorString(err)`	Error code to string.
`cudaGetErrorName(err)`	Error code to name.

Error checking pattern:

```
cudaError_t err = cudaMalloc(&ptr, size);
if (err != cudaSuccess) {
    printf("Error: %s\n", cudaGetErrorString(err));
}
// After kernel launch (async, doesn't return error):
kernel<<<grid, block>>>();
err = cudaGetLastError();       // Check launch error
err = cudaDeviceSynchronize();  // Check execution error
```

NVCC Compiler Options

Option	Description
`-arch=sm_XX`	Target GPU architecture (e.g., `sm_86`).
`-gencode=arch=compute_XX,code=sm_XX`	Generate code for specific arch.
`-G`	Debug mode; disables optimizations.
`-lineinfo`	Include line info for profiling.
`-O0/-O1/-O2/-O3`	Optimization level.
`-Xptxas -v`	Print register/memory usage per kernel.
`-maxrregcount=N`	Limit registers per thread.
`--ptxas-options=-v`	Verbose PTX assembler output.
`-res-usage`	Report resource usage.
`-Xcompiler "opts"`	Pass options to host compiler.

Example compilation:

```
nvcc -arch=sm_86 -O3 -Xptxas -v -o prog prog.cu
```